

Assignment 1: Replicating a Top-10 Solution

2nd Place, Kaggle *PhysioNet – Digitization of ECG Images*

Darren Sheehan : 122351591

20th October 2026

Contents

1 Overview **1**

2 Changes from the Author's Solution **2**

3 Leaderboard Submissions **5**

4 Training Runs **7**

5 Modifications Attempted **8**

1 Overview

The goal of this assignment is to replicate a solution from a past Kaggle competition. The solution replicated needs to have placed in the top 10 of the leaderboard to be eligible. The competition selected was *PhysioNet – Digitization of ECG Images* [1] and the solution replicated was from Takashi Someya (Kaggle Competitions Grandmaster) who placed 2nd. His solution was selected because of its detailed documentation [2] and excellent leaderboard performance (23.37 public / 23.27 private LB). This assignments replication solution uses the author's public inference notebook [3] and also his public training code [4]. The competition itself closed for leaderboard submission on the 22nd of January 2026, but it allows for "Late Submissions" which give both a public and private leaderboard score to allow for comparison with the author's original work.

The results displayed in this assignment will be used to help write the laboratory report. In addition to results from this assignment, the laboratory report will deal with the more technical understanding behind the authors solution and explain how different parts of the model work and also importantly how the provided kaggle data is used.

Labels notations used in this document are as follows: S1–S6 are submissions to Kaggle's leaderboard (table 5), R0–R2 are training runs (table 6), F, A and D are changes from the author's solution (section 2), and P and M are post-processing and training modifications respectively (section 5).

Table 1: Shared Kaggle notebooks and code. Each notebook name links to it on Kaggle (full name `darren-sheehan-<name>`); notebook 01 links to all the others.

Notebook (`<name>`)	What it is
Code repository (fork)	`https://github.com/darren1sheehan/physionet-image-digitization-Assignment-1`
`01-baseline-2nd-place-sub-cloned`	**Inference Notebook.** S1: Unchanged version of the author's original inference notebook
`03-baseline-training`	**Training.** R1: Replicating the original training script to train the Whole B0 model, over 12 epochs
`04-own-weights-submission`	S2: Using the Whole B0 model (R1's) weights with the author's inference script
`05-author-ensemble-p1`	S1+P1: lead-consistency post-processing
`07-mini-baseline`, `-m1`, `-m2`, `-m3`	R2 and training modifications M1–M3, 6 epochs each
`08-postprocessing-evaluation`	Offline evaluation of post-processing and decoders on fold 0
`09-own-weights-window-p1b`	S2+P2+P1b: window decoder and offset-safe lead consistency
`10-b6-weights-submission`	S4: R2's weights (baseline for S3)
`11-m3-weights-submission`	S3: M3's weights
`12-m3-12ep`	M3 trained for 12 epochs, as R1
`14-author-series-weight-2`	S5: the author's ensemble with re-weighted models (P3)
`15-m3-12ep-weights-submission`	S6: M3-12's weights (12 epochs, SNR-aware loss)

2 Changes from the Author's Solution

This section documents all the differences between this assignments replication and the original author's published code on GitHub [3, 4] and setup [2], with the justification for these changes as well. The GitHub fork listed in table 1 documents these code changes.

2.1 Fixes that were required for the code to run as the author intended

Table 2: Fixes to the public training repository.

	Change	Justification
F1	Rectified ECG pages were resized by the inference factor 5000/1962 (which resizes to 5607 px, and then is cropped to 5600 px) rather than a straight resize to 5600 px (`resize_rectified` in `transforms.py`)	Both the author's masks and the inference use the 5000/1962 scaling. This would have led to a 4–7 px drift in training at the image's right edge compared to the targets.
F2	16-bit training (mixed-precision) was only used on GPU (on for GPU, off for CPU) and `cuda` was used to refer to the GPU.	The CPU would run 125× slower using 16-bit (bf16) compared to 32-bit (fp32), also the name `cuda:1` kept crashing Kaggle's second GPU.
F3	Pin an older version of the `timm` library below version 1.0.29	Newer versions of timm rejects the the model names which the author has set in his configurations and saved models.
F4	Ground Truth ECG signals had to be loaded from the competition `train/{id}/{id}.csv`	The authors copy of this dataset was private
F5	`load_checkpoint` passes `weights_only=False`	PyTorch ≥ 2.6 loads with `weights_only=True` by default, which can reject the non-tensor values stored in the checkpoint (e.g. a NumPy best score), breaking `--resume`

2.2 Additions due to limited compute availability

Table 3: Training modifications added to the repository fork.

	Change	Reason
A1	Training now will use both of Kaggle's GPUs as opposed to just one (DistributedDataParallel started with `torchrun`). The image batch size remained at 4 so each GPU will get 2 images, validation is split between these GPUs.	In the author's forked original code setup just one GPU was used, but Kaggle gives access to two T4 GPUs. A single epoch running on one GPU took 1.87 h which over 12 epochs would eat into the already limited hours of compute.
A2	`--resume`: allows one to continue training from `last.pth`, including model weights, optimizer state, scheduler and AMP scaler state	Kaggle sessions have a maximum continuous runtime of 12 hours, this is needed for runs that took more than a single Kaggle session
A3	`training.max_hours`: ensures that training will stop before starting an epoch that will overrun the time allowance.	When a session is killed abruptly the output files can be lost, this earlier stopping ensures `last.pth` is preserved
A4	`history.csv`: during training one row per epoch is saved with the recorded learning rate, training and validation loss, validation SNR and the epoch's duration.	The author originally logged his training with "Weights & Biases", which is an online service. This CSV keeps the same record used for training curves, epoch times and overfitting checks.
A5	The Kaggle notebooks point to attached datasets on Kaggle and also contain shortcuts to the "ground truth" ECG signals for validation	The data amounts to approximately (62 GB) which is impractical to download for each Kaggle session.

2.3 Reduction in Model Scale for Training Replication

Table 4: Differences in training setup: author versus the assignment replication.

	Author	This replication	Reason
D1	Ensemble of 2 Whole + 4 Series models	Single Whole model	Compute budget of about 45 T4 GPU hours per week
D2	EfficientNet-B7 / V2-L / B6 encoders	EfficientNet-B0, default model on the public repository	The B7 or V2-L are large models, estimated 375–900 T4 GPU hours for 50 epochs, and also potentially not fit in Kaggle's 16 GB of VRAM
D3	50 epochs	12 epochs with cosine learning rate shortened to run over 12 instead of 50 epochs.	50 epochs of B0 model would still consume 94 h of GPU.
D4	Added 20,000 synthetic images in training	Only provided competition images used	This would cause each epoch to 3x in duration.
D5	Several folds and variants attempted	Training done on folds 1-4 and fold 0 was used for validation	Limited GPU compute

2.4 Inference and submission

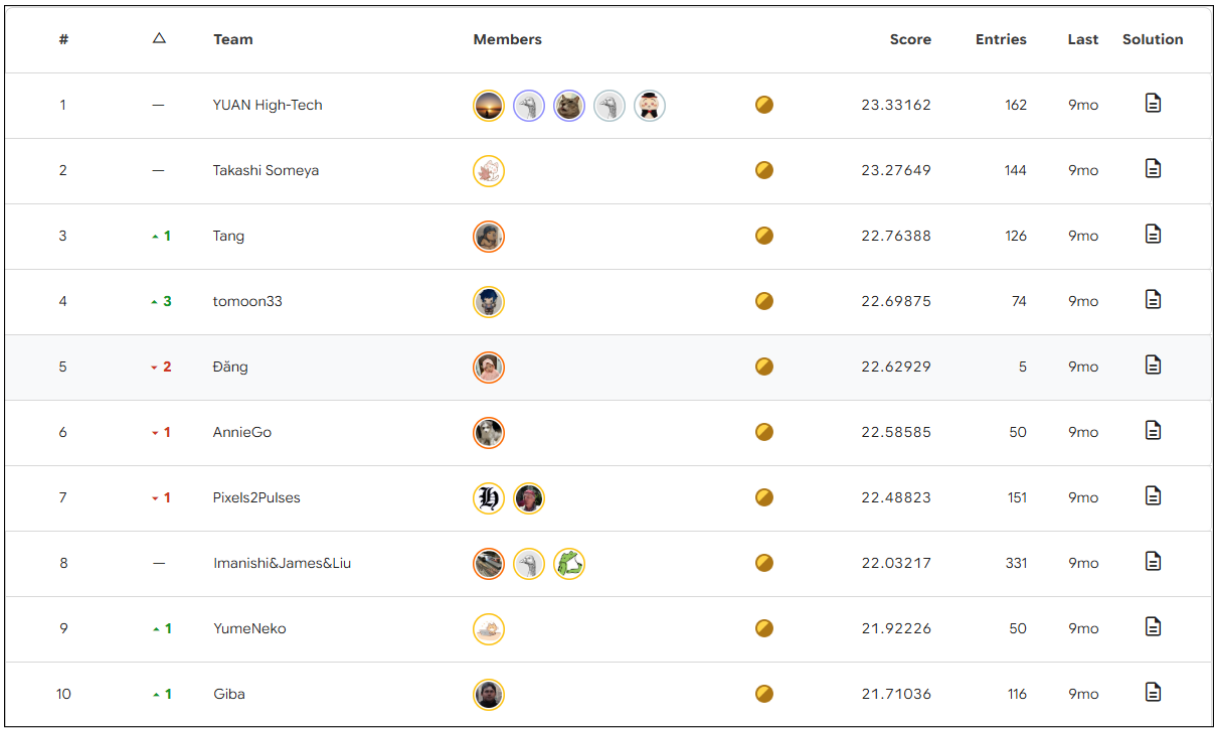
- **Submission 1 (S1)**: The author's notebook, inputs and pinned environment, unchanged.
- **Submissions 2 & 3 (S2, S3)**: These submissions use the model weights obtained from the recreation of the solutions training steps. Both submissions still utilise the S1 inference notebook. The only difference being that these submissions use just one model which is the Whole EfficientNet-B0 as opposed to a 6-model ensemble of both Whole and Series models. S2 is the baseline model trained for 12 epochs and S3 is trained for 6 epochs but has a modification (M3, table 8) to add SNR-aware loss.
- **Submission 4 (S4)**: This submission uses the same baseline model and setup as S3 training for the same 6 epochs, the difference with this submission is no M3 (SNR-aware loss). This submission acts as a control to compare the impact of M3 on the submission score.
- **Submission 5 (S5)**: This submission uses the full 6-model ensemble just like that of S1, in an attempt to try and improve the submission score a modification was made to how the model averaging was done. Series models were weighted twice as much as whole models in contrast to the original notebook which applied equal weighting across all six models in the ensemble.
- **Submission 6 (S6)**: This submission uses the S2 model and trains for the same number of epochs (12). The variation between S2 and S6 is just the addition of SNR-aware loss (M3, table 8). This makes allows for the comparison of the impact of the M3 modification after the full number of epochs.
- **Submission 1 with Post-Processing Modification (S1+P1)**: This takes the same model and weights from submission 1 and applies post processing modification (P1, table 8). This modification takes the models predictions and makes them consistent known relationships between the ECG leads (II = I + III and aVR + aVL + aVF = 0).
- **Submission 2 with Inference Modification's (S2+P2+P1b)**: This submission uses the same model weights as S2, it applies two inference changes. The first modification

(P2) is a window decoder, this takes the models signal output using the weighted average around its peak. The second modification (P1b) is simply a slightly altered version of the previously discussed post processing modification with the difference being that now the offset of the leads is first removed before the ECG lead relationships are applied.

3 Leaderboard Submissions

Table 5: Submissions (dB, the higher the score the better).

	Submission	Public	Private
–	Author's final ensemble, as reported [2]	23.37	23.27
Author's weights (6-model ensemble)			
S1	Unchanged re-run of the original notebook and weights	23.37	23.28
S1+P1	S1, with post-processing modification (P1)	18.16	18.41
S5	S1, with Series models weighted 2x as much as Whole models (P3)	**23.39**	**23.28**
Trained weights (Whole EfficientNet-B0)			
S2	12 epochs baseline	20.23	20.21
S2+P2+P1b	S2, with the window decoder (P2) and offset-safe lead consistency (P1b)	16.72	16.91
S3	6 epoch model with SNR-aware loss (M3)	19.16	19.14
S4	6 epochs baseline	17.98	17.67
S6	12 epoch model with SNR-aware loss (M3)	**20.56**	**20.48**



#	△	Team	Members	Score	Entries	Last	Solution
1	—	YUAN High-Tech		23.33162	162	9mo	
2	—	Takashi Someya		23.27649	144	9mo	
3	▲ 1	Tang		22.76388	126	9mo	
4	▲ 3	tomoon33		22.69875	74	9mo	
5	▼ 2	Đăng		22.62929	5	9mo	
6	▼ 1	AnnieGo		22.58585	50	9mo	
7	▼ 1	Pixels2Pulses		22.48823	151	9mo	
8	—	Imanishi&James&Liu		22.03217	331	9mo	
9	▲ 1	YumeNeko		21.92226	50	9mo	
10	▲ 1	Giba		21.71036	116	9mo	

Figure 1: The private Kaggle leaderboard, with Takashi Someya's 2nd-place submission at 23.27649.

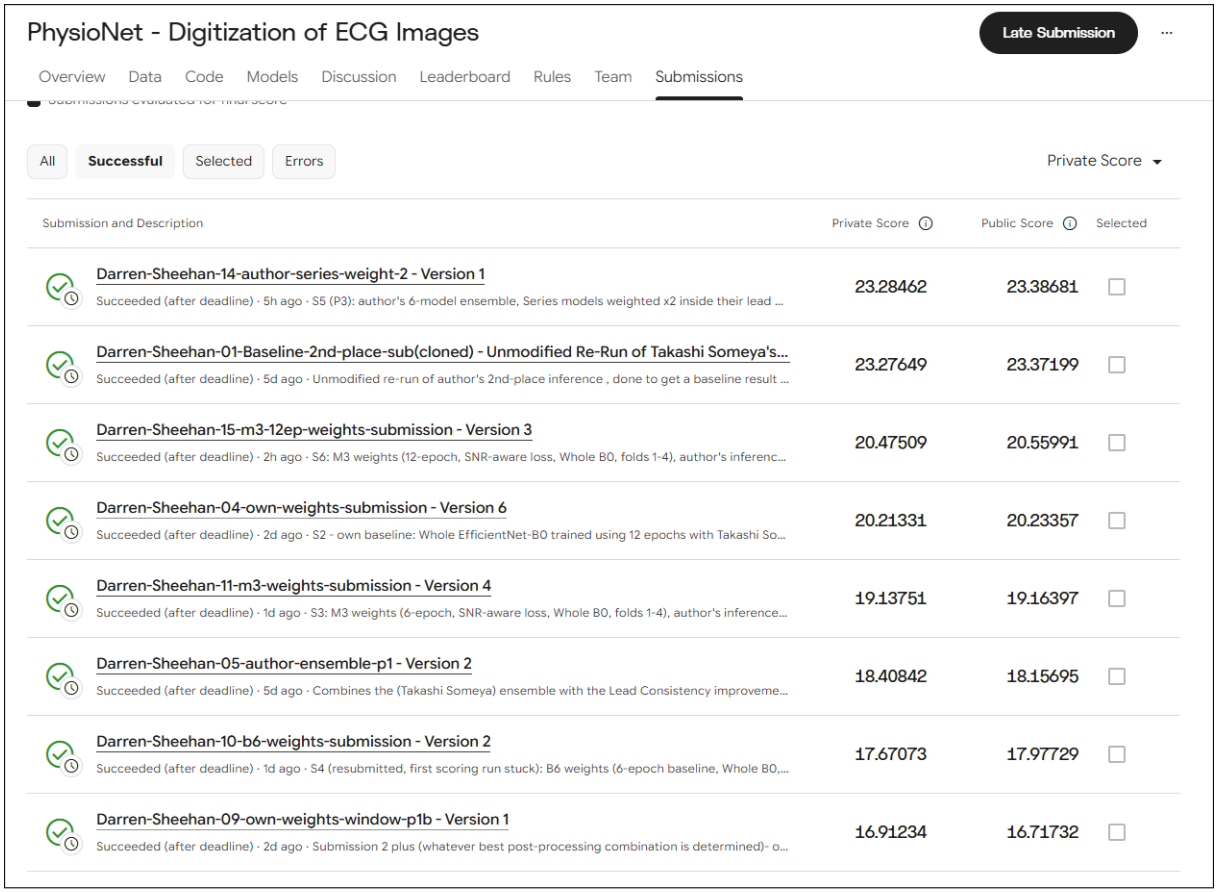


Figure 2: All Kaggle submissions for this assignment.

4 Training Runs

The original solution replicated trained 2 Whole and 4 Series models with large encoders (EfficientNet-B6/B7, EfficientNetV2-L) for 50 epochs, while also adding about 20,000 extra synthetic images. The level of compute required to completely replicate this training setup was not feasible for this assignment. The solution to this was to train a scaled down version.

Table 6: Training runs: All runs use folds 1–4 for training and fold 0 is held out for validation.

Run	Configuration	Epochs	Time	Val. SNR
R1	Baseline: Whole EfficientNet-B0, folds 1–4	12	10.9 h	23.24
R2	Baseline (6-epoch): R1 but less epochs	6	∼5.9 h	17.48
M1	R2, images resized as done in the public repository (without fix F1)	6	∼5.9 h	5.73
M2	R2, using also grayscale augmentation	6	∼5.6 h	17.39
M3	R2, with the SNR-aware loss	6	∼5.5 h	20.96
M3-12	R1 (12 epochs), with the SNR-aware loss	12	11.7 h	23.67

The run R1 was trained for 12 epochs with each epoch taking around 56 minutes using two T4 GPUs. As expected the 12 hour Kaggle time limit was reached after the 11th epoch, the `--resume` change allowed the saved checkpoint to simply be uploaded in order to continue the 12th epoch training. The last epoch barely added any gain (+0.03 dB) reaching a final validation SNR of 23.24 dB. This score is about 3 dB higher than S2's leaderboard score, which possibly suggests that Kaggle's hidden test images are more challenging than those in the validation fold.

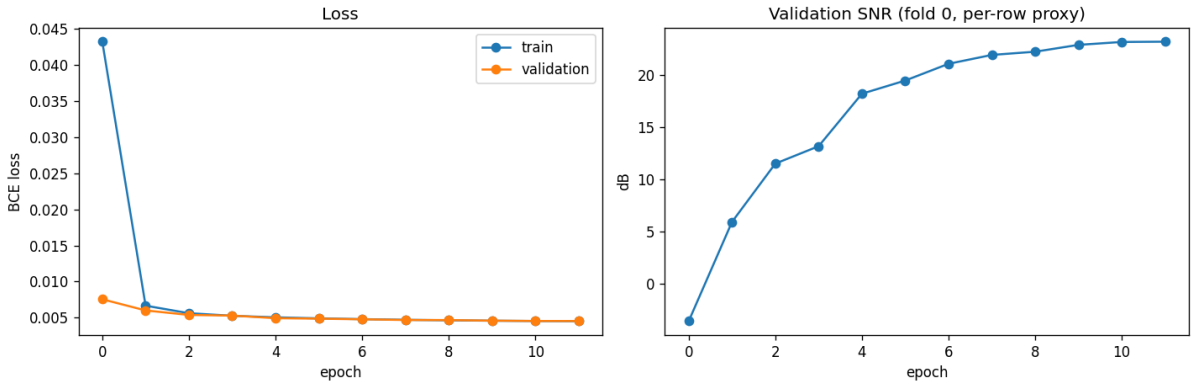


Figure 3: The training curves of R1. Left: Binary Cross-Entropy (BCE) loss. Right: Validation SNR on fold 0.

Table 7: Per-epoch history of R1 (`history.csv`).

Epoch	Learning rate	Train loss	Val. loss	Val. SNR (dB)
0	9.8×10^{-4}	0.0432	0.00756	−3.63
1	9.3×10^{-4}	0.0067	0.00601	5.91
2	8.5×10^{-4}	0.0056	0.00539	11.52
3	7.5×10^{-4}	0.0053	0.00529	13.19
4	6.3×10^{-4}	0.0051	0.00494	18.23
5	5.0×10^{-4}	0.0049	0.00487	19.50
6	3.7×10^{-4}	0.0048	0.00478	21.11
7	2.5×10^{-4}	0.0047	0.00469	21.96
8	1.5×10^{-4}	0.0046	0.00467	22.27
9	6.7×10^{-5}	0.0046	0.00459	22.93
10	1.7×10^{-5}	0.0045	0.00455	23.21
11	0	0.0045	0.00454	23.24

5 Modifications Attempted

Two types of modifications were carried out. *Post-processing* modifications (P), these keep the trained model weights and alter how the output of the model is turned into ECG signals. *Training* modifications (M) apply changes to the methods used to train the model itself. Many ideas which were explored took inspiration from other top solutions on the Kaggle leaderboard. Table 8 lists these modifications, and table 9 reports whether they improved the score or not.

Table 8: Modifications of the author's solution.

	Change	Source of the idea
P1	Same model weights: the two lead II predictions were averaged over 0–2.5 s (rhythm strip only in the original), then enforce II = I + III and aVR + aVL + aVF = 0	Used by 1st and 3rd place; 11th place provided evidence of it lowering SNR.
P3	Weight the ensemble, Series model prediction weighted 2x as much as Whole model	Own idea: the author gives single-model scores in his report, Series range 22.85–23.10 and Whole range 22.60–22.93
M1	`data.resize_mode=legacy`: training images resized as in the original repository	Own idea: Inspired by the the 4–7 px image–drift that was fixed by (F1)
M2	`data.grayscale_p=0.25`: grayscale augmentation	Listed as an augmentation in the write-up, but is absent from the repo missing from his code; Similar to 1st and 3rd place's
M3	`model.snr_loss_weight=0.1`: SNR-aware loss on the decoded signal, was added to BCE	11th place (+1.6 dB reported)

Testing the Modifications. To ensure that the modifications impacts were being tested fairly they were compared with a run with the same amount of training epochs and only differ by one unique change. Post-processing modifications were less computationally heavy and results were compared by the Kaggle leaderboard score. Training modifications were much more computationally demanding and as a result it was decided that the first 6 epoch training runs would be used to test whether a method had potential or not. These 6-epoch modifications were then compared against the baseline R2 result. M3 performed well over 6 epochs so it was also trained for the full 12 epochs and compared with R1. Validation SNR is defined in the training code as the estimate on fold 0; the leaderboard uses the kaggle competition's hidden test set and

own scoring.

Table 9: Modification Results (the higher the dB the better). Leaderboard scores are public / private.

	Compared	Measured on	Without	With	Change
P1	S1 vs S1+P1	Leaderboard	23.37 / 23.28	18.16 / 18.41	−5.2 / −4.9
P2+P1b	S2 vs S2+P2+P1b	Leaderboard	20.23 / 20.21	16.72 / 16.91	−3.5 / −3.3
P3	S1 vs S5	Leaderboard	23.37 / 23.28	23.39 / 23.28	+0.01 / +0.01
M1	R2 vs M1 (6 epochs)	Validation	17.48	5.73	−11.75
M2	R2 vs M2 (6 epochs)	Validation	17.48	17.39	−0.09
M3	R2 vs M3 (6 epochs)	Validation	17.48	20.96	+3.48
	S4 vs S3 (6 epochs)	Leaderboard	17.98 / 17.67	19.16 / 19.14	+1.19 / +1.47
	R1 vs M3-12 (12 epochs)	Validation	23.24	23.67	+0.43
	S2 vs S6 (12 epochs)	Leaderboard	20.23 / 20.21	20.56 / 20.48	+0.33 / +0.26

Post-processing (P1, P2+P1b). 1st and 3rd place both use this post processing method in their solutions, when tested using the validation data it resulted in a slight gain equal to 0.2 dB. It was found after running this modification on the Kaggle leaderboard it yielded a private score value approximately 5 dB's lower than the 2nd place score by the author. When using the off-set safe version (P1b) in combination with 3rd places window decoder (P2) it was found that a similar validation score was achieved but the leaderboard score of -3.4 dB.

Ensemble weighting (P3). The weighting of the series model 2x as much as the whole models (S5) for the ensembled model was able to actually improve upon the authors best result, although the gain was very small at approximately 0.01 dB giving it a private leaderboard score equal to 23.28 dB.

Resize (M1). Without the F1 fix validation was lowered by 11.75 dB: this is due to the model learning from the small 4-7px offset between images and their targets which results in the signal being incorrectly placed on correctly resized images. This confirms that fix F1 is essential for achieving the best SNR.

Grayscale augmentation (M2). Adding the grayscale augmentation mentioned in the author's write-up made no significant difference to the SNR results (−0.09 dB).

SNR-aware loss (M3). This modification was the only one found that clearly helped improve SNR by a significant amount. After the 6 epochs run validation score improved by 3.48 dB and the leaderboard score improved by 1.19 / 1.47 dB (S3 against S4). This is relatively near the +1.6 dB gain reported by the 11th place team who this modification was inspired by. It was observed that the majority of this gain comes early on in training which means that M3 learns much faster, with the full 12 epochs the validation gain shrinks to +0.43 dB. (S6) runs on the Kaggle leaderboard and managed to achieve a score of 20.48 dB on the private leaderboard. The SNR-aware loss still is a clear leaderboard improvement after the 12 epoch training and is the best result obtained by a model using my own training weights.

References

[1] PhysioNet – digitization of ECG images. Kaggle competition, 2025. URL https://www.kaggle.com/competitions/physionet-ecg-image-digitization.

[2] Takashi Someya. 2nd place solution – PhysioNet – digitization of ECG images. Kaggle write-up, 2026. URL https://www.kaggle.com/competitions/physionet-ecg-image-digitization/writeups/2nd-place-solution.

[3] Takashi Someya. physionet-2nd-place-submission. Kaggle notebook, 2026. URL https://www.kaggle.com/code/takashisomeya/physionet-2nd-place-submission.

[4] Takashi Someya. physionet-image-digitization (training code). GitHub repository, 2026. URL https://github.com/someya-takashi/physionet-image-digitization.